T.C.
AYDIN ADNAN MENDERES ÜNİVERSİTESİ
AYDIN MESLEK YÜKSEKOKULU
BİLGİSAYAR PROGRAMCILIĞI BÖLÜMÜ

WEB PROGRAMCILIĞI III
Nesne Temelli Programlama (OOP) Prensipleri Araştırma Ödevi

Öğrenci Adı Soyadı: Volkan KAYA
Öğrenci Numarası: 256001075
Öğretim Görevlisi: Öğr. Gör. Mehmet Can Hanaylı

---

1. Giriş: Nesne Temelli Programlama (OOP) Nedir ve Neden Kullanılır?

Bu araştırmada Nesne Temelli Programlama (Object-Oriented Programming - OOP) yaklaşımını, ortaya çıkış nedenlerini ve temel mantığını inceledim. Klasik yordamsal (prosedürel) programlama mantığında veriler ve fonksiyonlar birbirinden tamamen ayrı tanımlanıyordu. Küçük projelerde bu sorun yaratmasa da yazılan program büyüdükçe binlerce satırlık kodların takibi zorlaşıyor, bir fonksiyonu düzeltirken projenin başka yerleri bozulabiliyordu (spagetti kod).

Nesne Temelli Programlama, gerçek dünyadaki nesnelerin mantığını bilgisayar ortamına taşımıştır. Gerçek hayattaki bir arabanın rengi ve modeli gibi özellikleri ile çalışması veya fren yapması gibi eylemleri bir bütündür. OOP de değişkenleri (özellikleri) ve o değişkenlerle işlem yapan fonksiyonları (metotları) bir arada tutarak kodun çok daha derli toplu, hatasız ve tekrar kullanılabilir olmasını sağlar.

2. Sınıf (Class) ve Nesne (Object) Kavramları

Sınıf ve nesne, OOP'nin en temel iki kavramıdır ve aralarında plan ile inşa edilen ürün ilişkisi vardır:
• Sınıf (Class): Bir nesnenin hangi özelliklere ve işlevlere sahip olacağını belirten şablondur. Tek başına bellekte yer kaplamaz, sadece bir taslaktır.
• Nesne (Object): Bu şablondan 'new' komutu ile üretilen gerçek varlıktır. Bellekte (RAM) yer tutar ve içine veriler yazılır.

Çözdüğü Problem: Sınıf mantığı olmasaydı okuldaki 100 öğrenci için ayrı ayrı ogrenci1_ad, ogrenci2_ad gibi yüzlerce dağınık değişken tanımlamak gerekirdi. Sınıf sayesinde tüm bu verileri tek bir düzenli yapı altında toplayabiliyoruz.

Günlük Hayat Örneği: Mimarın çizdiği bina projesi bir sınıftır (class). Bu projeye bakılarak arsaya tuğlayla, çimentoyla yapılan ve içine girip oturulan gerçek bina ise nesnedir (object). Tek bir projeden yüzlerce ev yapılabilir.

```
// Sınıf tanımı (Kalıp)
public class Ogrenci
{
    public string Ad;
    public string Numara;

    public void BilgiYazdir()
    {
        Console.WriteLine("Öğrenci: " + Ad + " - No: " + Numara);
    }
}

// Nesne üretimi (RAM'de somut örnek)
Ogrenci ogr = new Ogrenci();
ogr.Ad = "Volkan";
ogr.Numara = "256001075";
ogr.BilgiYazdir();
```

3. Constructor (Yapıcı Metot) ve Nesne Oluşturma Mantığı

Constructor, bir sınıftan 'new' anahtar kelimesiyle nesne oluşturduğumuz anda ilk çalışan özel metottur. Adı sınıf adıyla tamamen aynı olmak zorundadır ve geri dönüş tipi (void dahi) yazılmaz.

Neden Kullanılır ve Çözdüğü Problem: Bir nesne oluşturulurken bazı alanların kesinlikle boş bırakılmaması gerekir. Örneğin bir banka hesabı açılırken hesap numarası veya sahibinin adı girilmezse o nesne eksik kalır ve sistem çöker. Constructor kullanmadığımızda programcı bu alanları doldurmayı unutabilir. Constructor ile zorunlu bilgileri parametre olarak isteyerek nesnenin baştan eksiksiz ve güvenli şekilde oluşturulmasını sağlarız.

Günlük Hayat Örneği: Fabrikadan çıkan bir arabanın tekerlekleri veya motoru takılmadan müşteriye teslim edilmemesi gibidir. Araba ancak çalışır durumda fabrikadan teslim edilir.

```
public class BankaHesabi
{
    public string HesapNo;
    public string Sahip;

    // Constructor: Nesne üretilirken bu bilgileri zorunlu kılar
    public BankaHesabi(string hesapNo, string sahip)
    {
        HesapNo = hesapNo;
        Sahip = sahip;
    }
}

// Kullanımı:
// BankaHesabi h = new BankaHesabi(); // HATA verir, parametresiz
```

```
açılamaz
BankaHesabi h = new BankaHesabi("TR12345", "Volkan Kaya");
```

4. Access Modifiers (Erişim Belirteçleri: public, private, protected)

Sınıf içindeki değişken ve metotlara dışarıdan kimlerin erişebileceğini belirleyen anahtar sözcüklerdir:

• public: Dışarıya tamamen açıktır. Hem sınıfın içinden hem de projenin diğer yerlerinden doğrudan erişilebilir.

• private: Tamamen gizlidir. Sadece tanımlandığı sınıfın içinden erişilebilir, dışarıdan kimse dokunamaz.

• protected: Sadece o sınıfta ve o sınıftan miras alan alt sınıflarda kullanılabilir, harici dış dünyaya kapalıdır.

Çözdüğü Problem: Sınıfın içindeki önemli verilerin dışarıdan bilinçsizce değiştirilmesini engeller. Örneğin hesap bakiyesini public yaparsak dışarıdan biri rastgele eksi bir bakiye atayabilir. Private yaparak bu riski önleriz.

Günlük Hayat Örneği: ATM makinesinde tuş takımı ve para çekme yuvası public'tir, herkes kullanabilir. Kasanın içindeki para çekmecesi private'tır, sadece ATM'nin kendi iç mekanizması ulaşabilir. Arkadaki servis kapağı ise protected'tır, sadece bankanın yetkili bakım görevlileri açabilir.

```
public class Atm
{
    public string Konum = "Aydın Kampüs"; // Dışarıdan görünür
    private decimal kasaParasi = 100000;  // Dışarıdan doğrudan
değiştirilemez

    public void ParaVer(decimal miktar)
    {
        if (miktar <= kasaParasi)
        {
            kasaParasi -= miktar;
            Console.WriteLine(miktar + " TL verildi.");
        }
    }
}
```

5. Encapsulation (Kapsülleme)

Kapsülleme, sınıfın alanlarını (değişkenlerini) private yaparak dış müdahalelerden gizlemek ve onlara erişimi Property (get ve set blokları) üzerinden kontrollü bir biçimde sağlamaktır.

Çözdüğü Problem: Hatalı ve mantıksız veri girişini önler. Eğer bir kişinin yaşı doğrudan public olsaydı birisi 'yas = -25' yazabilirdi. Kapsülleme ile 'set' bloğunun içine bir if şartı koyarak negatif yaş girilmesini engelleriz. Böylece verinin önüne bir güvenlik filtresi koymuş oluruz.

Günlük Hayat Örneği: Eczaneden aldığımız kapsül ilaçlar gibidir. İlacın içindeki kimyasal toza doğrudan elimizle dokunmayız, kapsül kabuğu onu dış etkilerden korur ve midede tam zamanında çözülmesini sağlar.

```
public class Kisi
{
    private int _yas; // Gizli değişken

    public int Yas
    {
        get { return _yas; }
        set
        {
            if (value > 0)
                _yas = value; // Sadece mantıklı değerleri kabul eder
            else
                Console.WriteLine("Hata: Yaş 0'dan büyük olmalıdır!");
        }
    }
}
```

6. Abstraction (Soyutlama)

Soyutlama, kullanıcının bilmesine gerek olmayan karmaşık iç detayları gizleyip, sadece kullanması gereken temel işlevi dışarıya sunmaktır. Yani işin 'nasıl yapıldığı' arka planda kalır, kullanıcı sadece 'ne yapıldığına' odaklanır.

Çözdüğü Problem: Kod kalabalığını ve bağımlılığı azaltır. Örneğin bir web sitesinde bankadan ödeme alırken arka plandaki şifreleme algoritmalarını veya bağlantı protokollerini her defasında bilmemiz gerekmez. Bize sadece 'OdemeYap(tutar)' şeklinde basit bir metot yeterlidir.

Günlük Hayat Örneği: Araba sürerken hızlanmak için gaza basarız. Gaza bastığımızda motora ne kadar yakıt püskürtüldüğünü veya subapların nasıl çalıştığını bilmemiz gerekmez. Gaz pedalı sürücü için motorun karmaşıklığını soyutlamıştır.

```
// Kullanıcı sadece bu sade sözleşmeyi bilir
public interface IOdemeServisi
{
    void OdemeYap(decimal tutar);
}

// Karmaşık banka kodları bu sınıfın içinde gizlidir
public class BankaServisi : IOdemeServisi
{
    public void OdemeYap(decimal tutar)
    {
        // Arka plandaki karmaşık banka API kodları
        Console.WriteLine(tutar + " TL tahsil edildi.");
    }
}
```

7. Inheritance (Kalıtım / Miras Alma)

Kalıtım, daha önce yazılmış bir sınıfın (Base Class) tüm özellik ve metotlarını yeni bir sınıfa (Derived Class) aktarabilmesidir. Sınıflar arasında 'bir türüdür' (IS-A) ilişkisi kurar.

Çözdüğü Problem: Kod tekrarını önler. Örneğin bir okul sisteminde Öğretmen ve Öğrenci sınıfları olsun. Her ikisinde de Ad, Soyad, TC No gibi ortak alanlar vardır. İki sınıfa da bunları ayrı ayrı yazmak yerine tek bir 'Kisi' sınıfı açıp oraya yazarız; Öğretmen ve Öğrenci sınıflarını Kisi'den miras alırız. Böylece hem kod kısalır hem de Kisi'de yapılan bir değişiklik diğer sınıflara otomatik geçer.

Günlük Hayat Örneği: Bir çocuğun anne babasından göz rengi veya boy gibi genetik özellikleri devralması gibidir.

```
// Ana Sınıf (Ortak alanlar)
public class Kisi
{
    public string AdSoyad;
    public string TcNo;
}

// Ogrenci bir Kisi'dir (Miras alır)
public class Ogrenci : Kisi
{
    public string OkulNo;
}

// Ogretmen bir Kisi'dir (Miras alır)
public class Ogretmen : Kisi
{
    public string Brans;
}
```

8. Polymorphism (Çok Biçimlilik)

Çok biçimlilik, aynı isimdeki bir metodun farklı nesnelerde farklı şekilde davranabilmesidir. Temel sınıfta 'virtual' olarak tanımlanan bir metot, alt sınıfta 'override' edilerek o sınıfa özel hale getirilir.

Çözdüğü Problem: Kodun içinde sürekli 'if (tur == KrediKarti) ... else if (tur == Havale) ...' gibi upuzun karar blokları yazma zorunluluğunu ortadan kaldırır. Yeni bir yöntem eklendiğinde eski kodları bozmadan sadece yeni bir sınıf yazmamız yeterli olur.

Günlük Hayat Örneği: Hayvanlara 'Ses Çıkar' komutu verdiğimizi düşünelim. Köpek havlar, kedi miyavlar. Komut aynıdır ama her hayvan kendi doğasına göre farklı ses verir.

```
public class Hayvan
{
    public virtual void SesCikar() => Console.WriteLine("Bir ses
çıktı.");
}

public class Kopek : Hayvan
{
    public override void SesCikar() => Console.WriteLine("Hav hav!");
}
```

```
public class Kedi : Hayvan
{
    public override void SesCikar() => Console.WriteLine("Miyav!");
}
```

9. Interface (Arayüz) ve Abstract Class (Soyut Sınıf)

Hem Interface hem de Abstract Class doğrudan 'new' ile üretilemeyen, alt sınıflara yol gösteren yapılardır:
• Abstract Class: Birbirine benzeyen, akraba sınıflar için temel bir çatı sağlar. İçinde hem kodları yazılmış normal metotlar hem de gövdesiz (abstract) metotlar bulunabilir. C#'ta bir sınıf sadece bir abstract sınıftan miras alabilir.
• Interface: Sadece kuralları bildirir, içinde hiçbir kod bulunmaz. Bir sözleşmedir; 'Beni kullanan sınıf bu metotları yazmak zorundadır' der. En büyük avantajı bir sınıfın birden fazla interface uygulayabilmesidir.

Hangisi Ne Zaman Seçilir? Ortak kod paylaşımı ve akrabalık varsa Abstract Class seçilir (örneğin Hayvan -> Kuş). Birbirinden çok farklı sınıflara ortak bir yetenek kazandırmak istiyorsak Interface seçilir (örneğin hem Fatura hem de Resim sınıfı IYazdirilabilir olabilir).

```
// Interface: Sadece kural bildirir
public interface IYazdirilabilir
{
    void Yazdir();
}

// Abstract Class: Hem gövdeli hem gövdesiz metot içerebilir
public abstract class Sekil
{
    public abstract double AlanHesapla(); // Alt sınıf kendi yazar
    public void BilgiVer() => Console.WriteLine("Geometrik şekil");
}
```

10. Composition (Kompozisyon) ve Inheritance (Kalıtım) Arasındaki Fark

OOP tasarımında sıkça vurgulanan 'Kalıtım yerine kompozisyonu tercih edin' kuralı aralarındaki farkı özetler:
• Inheritance (Kalıtım - IS-A): 'Otomobil bir Araçtır'. Sınıflar birbirine göbekten bağlanır. Üst sınıfta yapılan küçük bir değişiklik tüm alt sınıfları bozabilir.
• Composition (Kompozisyon - HAS-A): 'Otomobilin bir Motoru vardır'. Bir sınıf diğer sınıfı miras almak yerine onun bir nesnesini kendi içinde parça olarak barındırır.

Çözdüğü Problem: Kalıtım sınıfları çok bağımlı hale getirir. Kompozisyon sayesinde parçalar birbirinden bağımsız kalır. Örneğin yarın arabanın motorunu benzinliden elektriğe çevirmek istediğimizde arabayı baştan kodlamayız, sadece motor parçasını değiştiririz.

Günlük Hayat Örneği: Masaüstü bilgisayar toplamak kompozisyondur. Ekran kartı ve işlemci anakartın çocuğu değildir; anakarta takılıp çıkarılabilen bağımsız parçalardır.

```
public class Motor
{
```

```
    public void Calistir() => Console.WriteLine("Motor devrede.");
}

public class Araba
{
    // Kompozisyon: Araba motordan türemez, içinde Motor nesnesi
barındırır (HAS-A)
    private Motor _motor = new Motor();

    public void HareketEt()
    {
        _motor.Calistir();
        Console.WriteLine("Araba ilerliyor.");
    }
}
```

11. Nesneler Arasındaki İlişkiler (Özet)

Sınıflar projelerde birbirleriyle farklı şekillerde ilişki kurarlar:
1. Inheritance (Kalıtım - IS-A): Bir nesnenin diğerinin bir türü olmasıdır (Öğrenci bir İnsandır).
2. Composition (Kompozisyon - Sıkı Sahiplik): Parça bütüne tamamen bağımlıdır. Bütün yok edilirse parça da yok olur. Örneğin 'Ev ve Odalar'. Ev yıkılırsa içindeki odalar da yok olur, tek başına bir oda sokakta var olamaz.
3. Aggregation (Kümeleme - Gevşek Sahiplik): Parça-bütün ilişkisi vardır ama parça tek başına da yaşayabilir. Örneğin 'Bölüm ve Öğretim Görevlisi'. Bilgisayar bölümü kapansa bile hoca var olmaya devam eder.
4. Dependency (Bağımlılık - USES-A): En geçici ilişkidir. Bir sınıfın diğerini sadece bir metot içinde parametre olarak kullanıp bırakmasıdır. Örneğin Yazici sınıfının Yazdir(Belge b) metodunda belgeyi geçici olarak kullanması gibi.

```
// Dependency (USES-A): FaturaYazici, Siparis'i sadece geçici parametre
olarak kullanır
public class FaturaYazici
{
    public void FaturaBas(Siparis siparis)
    {
        Console.WriteLine(siparis.Tutar + " TL fatura basıldı.");
    }
}
```

12. Sonuç

Bu araştırma ödevinde Nesne Temelli Programlamanın sadece teorik kurallardan ibaret olmadığını, aslında gerçek hayatı yazılıma uyarlayarak işimizi kolaylaştıran bir mantık olduğunu gördüm. Kapsülleme ile verilerimizi hatalı girişlerden korurken, Kalıtım ile kod tekrarından kurtuluyoruz. Çok Biçimlilik ve Soyutlama sayesinde ise yazdığımız web projeleri daha esnek hale geliyor ve yeni özellikler eklerken eski kodları bozma korkusu yaşamıyoruz. Bu prensipler özellikle Web Programcılığı dersinde gördüğümüz veri tabanı modelleri, kullanıcı oturumları ve web servisleri geliştirirken temiz kod yazmanın temelidir.

13. Kaynakça

Bu araştırma ödevi hazırlanırken aşağıdaki kaynaklardan, kitaplardan ve resmi teknik dokümantasyonlardan yararlanılmıştır:

• **Microsoft Learn.** "C# Dokümantasyonu: Nesne Yönelimli Programlama (OOP) Prensipleri ve Sınıf Mimarisi". Erişim adresi: https://learn.microsoft.com/tr-tr/dotnet/csharp/fundamentals/object-oriented/

• **Aslan, M. (2020).** A'dan Z'ye C# ile Nesne Yönelimli Programlama ve Yazılım Mimarisi (4. Baskı). İstanbul: Pusula Yayıncılık.

• **Karagülle, İ. ve Pala, Z. (2018).** Nesne Yönelimli Programlama ve C# (3. Baskı). İstanbul: Türkmen Kitabevi.

• **W3Schools.** "C# OOP (Object-Oriented Programming) Tutorial & Class Concepts". Erişim adresi: https://www.w3schools.com/cs/cs_oop.php

• **Troelsen, A. ve Japikse, P. (2021).** Pro C# 9 with .NET 5: Foundational Principles and Practices in Programming (10th ed.). Apress Yayınları.